

Modelos Generativos Profundos

Clase 11: Redes generativas adversarias (parte I)

Fernando Fêtis Riquelme

Otoño, 2026

Facultad de Ciencias Físicas y Matemáticas
Universidad de Chile

Revisión Tarea 1

Formulación de una GAN

Generación condicional

Heurísticas básicas para GANs

Revisión Tarea 1

Formulación de una GAN

Idea general

Una red **generativa adversaria (GAN)** es una red bayesiana de variable latente, $p_{\theta}(x, z) = p(z)p_{\theta}(x|z)$, entrenada buscando *engañar* a un clasificador $q_{\phi}(y|x)$ que aprende a diferenciar:

- **Muestras reales** ($y = 1$) provenientes del conjunto de entrenamiento, i.e., de la distribución $p_{\text{data}}(x)$.
- **Muestras generadas** ($y = 0$) provenientes del modelo $p_{\theta}(x)$.

Ambas redes se entrenan de manera adversativa, compartiendo una misma función objetivo, formando un **juego de suma cero**.

Modelos probabilísticos de una GAN

Una GAN considera las siguientes distribuciones:

- **Prior sobre el espacio latente:** $z \sim \mathcal{N}(0, I_L)$.
- **Generador:** $p_\theta(x|z) = \delta_{G_\theta(z)}(x)$ (i.e., es determinista: $z \mapsto G_\theta(z)$), donde $G_\theta : \mathbb{R}^L \rightarrow \mathbb{R}^D$ es una red neuronal.
- **Discriminador:** $q_\phi(y|x) \sim \text{Bernoulli}(D_\phi(x))$ es un clasificador binario, donde $D_\phi : \mathbb{R}^D \rightarrow [0, 1]$ es una red neuronal.

Función objetivo

Dada la descripción anterior, una GAN es entrenada para optimizar el siguiente problema minimax:

$$\min_{\theta} \max_{\phi} \mathcal{L}_{\text{GAN}}(\theta, \phi),$$

donde $\mathcal{L}_{\text{GAN}}(\theta, \phi)$ es la log-verosimilitud esperada del clasificador sobre una distribución conjunta (x, y) formada por muestras reales y falsas:

$$\mathcal{L}_{\text{GAN}}(\theta, \phi) = \mathbb{E}_{(x,y)} [\log q_{\phi}(y|x)]$$

Considerando que $q_\phi(y|x) \sim \text{Bernoulli}(D_\phi(x))$, entonces:

$$q_\phi(y|x) = \begin{cases} D_\phi(x) & \text{si } y = 1 \\ 1 - D_\phi(x) & \text{si } y = 0 \end{cases} = D_\phi(x)^y (1 - D_\phi(x))^{1-y},$$

por lo que la log-verosimilitud del clasificador es

$$\log q_\phi(y|x) = y \log D_\phi(x) + (1 - y) \log (1 - D_\phi(x)),$$

donde se reconoce la **entropía cruzada binaria** entre las etiquetas reales y las predicciones del clasificador. Luego:

$$\mathcal{L}_{\text{GAN}}(\theta, \phi) = -\mathbb{E}_x [\text{CE}(p(y|x), q_\phi(y|x))]$$

Función de pérdida

Una GAN es entrenada utilizando una combinación (mixtura) de imágenes reales ($y = 1$) e imágenes generadas ($y = 0$):

$$(x, y) \sim \frac{1}{2} (p_{\text{data}}(x) \delta_1(y) + p_{\theta}(x) \delta_0(y))$$

Por lo tanto:

$$\begin{aligned}\mathcal{L}_{\text{GAN}}(\theta, \phi) &= \mathbb{E}_{(x, y)} [\log q_{\phi}(y|x)] \\ &= \frac{1}{2} (\mathbb{E}_{p_{\text{data}}(x)} [\log D_{\phi}(x)] + \mathbb{E}_{p_{\theta}(x)} [\log(1 - D_{\phi}(x))]) \\ &= \frac{1}{2} (\mathbb{E}_{p_{\text{data}}(x)} [\log D_{\phi}(x)] + \mathbb{E}_{p(z)} [\log(1 - D_{\phi}(G_{\theta}(z)))])\end{aligned}$$

donde se usó que $p_{\theta}(x|z) \sim \delta_{G_{\theta}(z)}(x)$ para $z \sim p(z)$.

Función de pérdida

Para efectos de implementación, ambas esperanzas se estiman con aproximaciones de Monte Carlo:

- $\mathbb{E}_{p_{\text{data}}(x)}[\cdots]$ se aproxima usando un conjunto de entrenamiento i.i.d. $\mathcal{D} = \{x^1, \dots, x^N\} \subset \mathbb{R}^D$ provenientes desde $p_{\text{data}}(x)$.
- $\mathbb{E}_{p(z)}[\cdots]$ se aproxima usando muestras i.i.d. $\{z^1, \dots, z^N\} \subset \mathbb{R}^L$ generadas desde $p(z) \sim \mathcal{N}(0, I_L)$.

Generación condicional

Se puede extender la formulación de una GAN para aprender distribuciones condicionales $p_{\text{data}}(x|c)$ mediante pares de entrenamiento de la forma $(x, c) \in \mathbb{R}^D \times \mathcal{C}$, donde $\mathcal{C}$ es el conjunto donde vive el factor condicionante del modelo.

En cualquiera de los casos, la red bayesiana de una **GAN condicional (CGAN)** es la siguiente:

$$p_{\theta}(x, z|c) = p(z)p_{\theta}(x|c, z),$$

donde el hecho de seguir usando un prior latente gaussiano $p(z) \sim \mathcal{N}(0, I_L)$, permite considerar que $z \perp c$.

Entrenamiento de una CGAN

En general, en una GAN condicional, el generador es una red neuronal $G_\theta : \mathcal{C} \times \mathbb{R}^L \rightarrow \mathbb{R}^D$, mientras que el discriminador es una red neuronal $D_\phi : \mathcal{C} \times \mathbb{R}^D \rightarrow [0, 1]$.

La función de pérdida adversativa, $\mathcal{L}_{\text{CGAN}}(\theta, \phi)$, en este caso es:

$$\mathbb{E}_{p_{\text{data}}(x, c)} [\log D_\phi(c, x)] + \mathbb{E}_{p(z)p_{\text{data}}(c)} [\log (1 - D_\phi(c, G_\theta(c, z)))],$$

donde $p_{\text{data}}(x, c)$ es representado por un dataset supervisado, mientras que $z \sim \mathcal{N}(0, I_L)$.

Condicionamiento con etiquetas

En el caso más simple, $c \in \{c_1, \dots, c_K\}$ puede ser una **etiqueta de clase**. En este caso:

- Se puede utilizar una representación one-hot de cada etiqueta.
- Se puede utilizar una matriz de embeddings para representar vectorialmente cada etiqueta.

En ambos casos, la representación vectorial de c se concatena a la entrada de G_θ y D_ϕ .

Condicionamiento con texto

También es posible considerar el caso donde $c \in \mathcal{V}^*$ es una **secuencia de tokens**. En este caso:

- Se puede utilizar un modelo tipo BERT (usando el token <CLS>) o CLIP para obtener una representación vectorial de c .
- Además, es posible utilizar ambos modelos a la vez, lo que permite inyectar representaciones de naturaleza semántica (BERT) y visual (CLIP) dentro la arquitectura neuronal.
- Otra opción posible es utilizar un mecanismo de cross attention para que el generador y el discriminador puedan acceder a la representación de cada token de la secuencia c .
- Este último enfoque de arquitectura se usa en algunos modelos de difusión como Stable Diffusion.

Condicionamiento con imágenes

Otra opción es considerar a c como una **imagen**. En este caso:

- Se puede utilizar un modelo tipo ViT (o tipo VAE) para obtener una representación vectorial de c .
- También es posible utilizar arquitecturas image-to-image que combinen convoluciones y convoluciones transpuestas.
- La arquitectura U-Net (adaptada con mecanismos de atención) ha sido un modelo importante para modelos de imagen.

Heurísticas básicas para GANs

Dificultades del entrenamiento

Una limitación característica del entrenamiento de una GAN es que es **inestable** y altamente sensible al balance entre generador y discriminador.

Por esto, la literatura de GANs ha desarrollado un conjunto de heurísticas que buscan estabilizar el entrenamiento tanto a nivel de arquitectura como de función de pérdida.

Algunas de estas heurísticas han motivado trabajos posteriores en modelos de difusión.

Balance entre generador y discriminador

El principal problema en el entrenamiento de una GAN es que el clasificador tiende a aprender más rápido que el generador.

Si D_ϕ se vuelve *demasiado bueno* diferenciando muestras reales y falsas, entonces $\log(1 - D_\phi(G_\theta(z)))$ se satura y G_θ deja de recibir gradientes útiles.

Se revisarán algunas heurísticas usuales para mitigar esto.

Una heurística simple consiste en actualizar G_θ más veces que D_ϕ , realizando $k > 1$ steps de optimización del generador por cada step del discriminador.

Esto permite que G_θ *alcance* a D_ϕ durante el entrenamiento, evitando que el discriminador se vuelva demasiado fuerte y deje de entregar gradientes útiles.

Label smoothing

Otra heurística consiste en utilizar **label smoothing** en las muestras reales. Para esto, en lugar de usar $y = 1$ en la pérdida del discriminador, se suele utilizar $y = 0.9$ (o un valor similar).

Esto regulariza a D_ϕ y evita que su salida se sature en 1, manteniendo gradientes útiles para G_θ .

Recordar que en los LLMs también se puede utilizar label smoothing, aunque por una motivación distinta (considerar que puede haber múltiples *próximos tokens* posibles).

Pérdida no saturante

Otra observación importante es con respecto al problema de optimización original del generador:

$$\min_{\theta} \mathbb{E}_{p(z)} [\log(1 - D_{\phi}(G_{\theta}(z)))]$$

Notar que la función objetivo tiene gradientes muy pequeños cuando $D_{\phi}(G_{\theta}(z)) \approx 0$ (i.e., al inicio del entrenamiento).

Debido a esto, en la práctica se utiliza una versión **no saturante**:

$$\max_{\theta} \mathbb{E}_{p(z)} [\log D_{\phi}(G_{\theta}(z))],$$

que corresponde a la maximización de la probabilidad de que el discriminador clasifique como verdadera a una muestra generada.

Modos de falla

Incluso aplicando las heurísticas anteriores, el entrenamiento de una GAN puede fallar de otras maneras, por ejemplo:

- **Mode collapse:** el generador produce muestras de baja diversidad por quedar atrapado en una región del espacio latente que es buena engañando al discriminador.
- **Vanishing gradients:** D_ϕ se vuelve demasiado bueno y G_θ no recibe gradientes útiles.

Esto ha motivado el desarrollo de arquitecturas y funciones de pérdida más sofisticadas como la **Wasserstein GAN**, la cual está relacionada al **transporte óptimo**. Este último campo se ha vuelto muy importante en el estudio de modelos generativos.

En la próxima clase:

- Arquitecturas neuronales para imágenes.
- Algunas GANs clásicas.

Modelos Generativos Profundos

Clase 11: Redes generativas adversarias (parte I)